1. Deney

Bulut Tabanlı AI Siber Güvenlik ve Yazılım Servislerinin Nicel Karşılaştırmalı Analizi

Deneyin Amacı:

Bu deneyin amacı; bir bulut servis sağlayıcısının sunduğu yapay zekâ, siber güvenlik ve yazılım geliştirme servislerinin:
• Doğru sınıflandırılması,
• Nicel metriklerle değerlendirilmesi,
• Karşılaştırılabilir çıktılar üretilmesidir.

Seçilen servis sağlayıcı AWS'dir.
Sebep: Kariyerime Spring odaklı devam etmek istediğim için Spring ekosistemi ile entegre olması kullanışlı bir bulut servis sağlayıcı seçmek istedim. Spring cloud agnostic olduğu için bütün büyük bulut servis sağlayıcılarla iyi çalışabiliyor bu yüzden seçimimi yapmamda devreye diğer etkenler girdi. Popülerliği ve endüstri standardı olması AWS'yi öne çıkarıyordu. Bunun yanı sıra bu ders kapsamındaki deneylerin neredeyse hepsini ücretsiz bir şekilde gerçekleştirebilmemi de sağlayacak. Sağladığı servislerin kataloğu büyük ve deneylerimde kullanabileceğim birçok seçenek sunuyor.

Görev–1: Servis Tanımlama ve Sınıflandırma Doğruluğu (20 Puan)

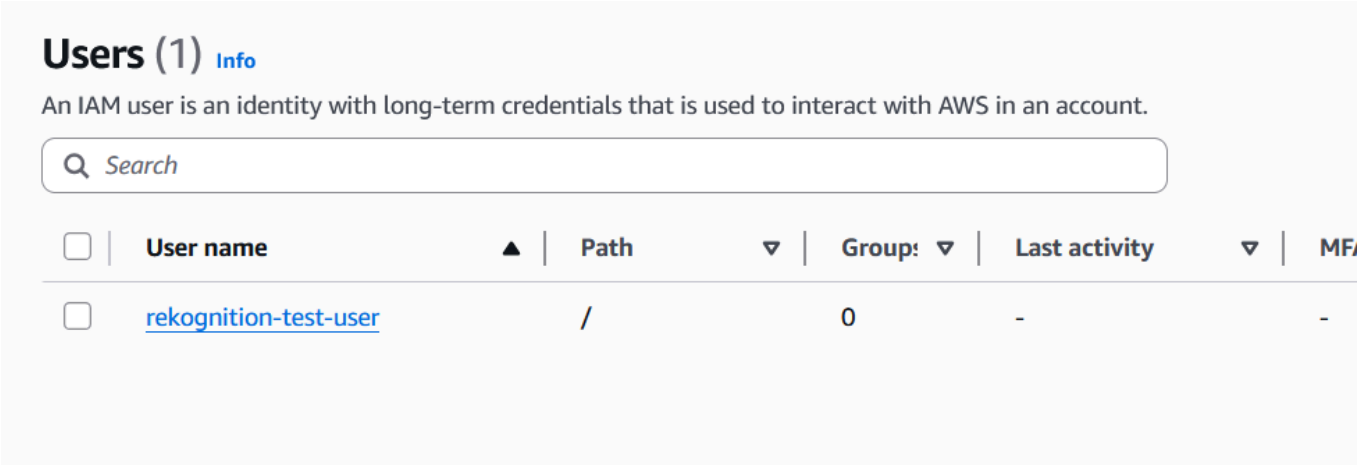
Tablo 1. Servis Tanımlama Tablosu

Servis Adı	Alan (AI / Güvenlik /Yazılım)	Servis Modeli (IaaS/PaaS/SaaS)	Doğru/Yanlış
Rekognition	AI	SaaS	
Amazon Comprehend	AI	SaaS	
IAM	Security	PaaS	
GuardDuty	Security	SaaS	
Elastic Beanstalk	Yazılım	PaaS	
CodePipeline	Yazılım	SaaS	

Görev–2: AI Servisleri Nicel Değerlendirme (25 Puan)

Rekognition:

1 -Test user oluşturuldu.

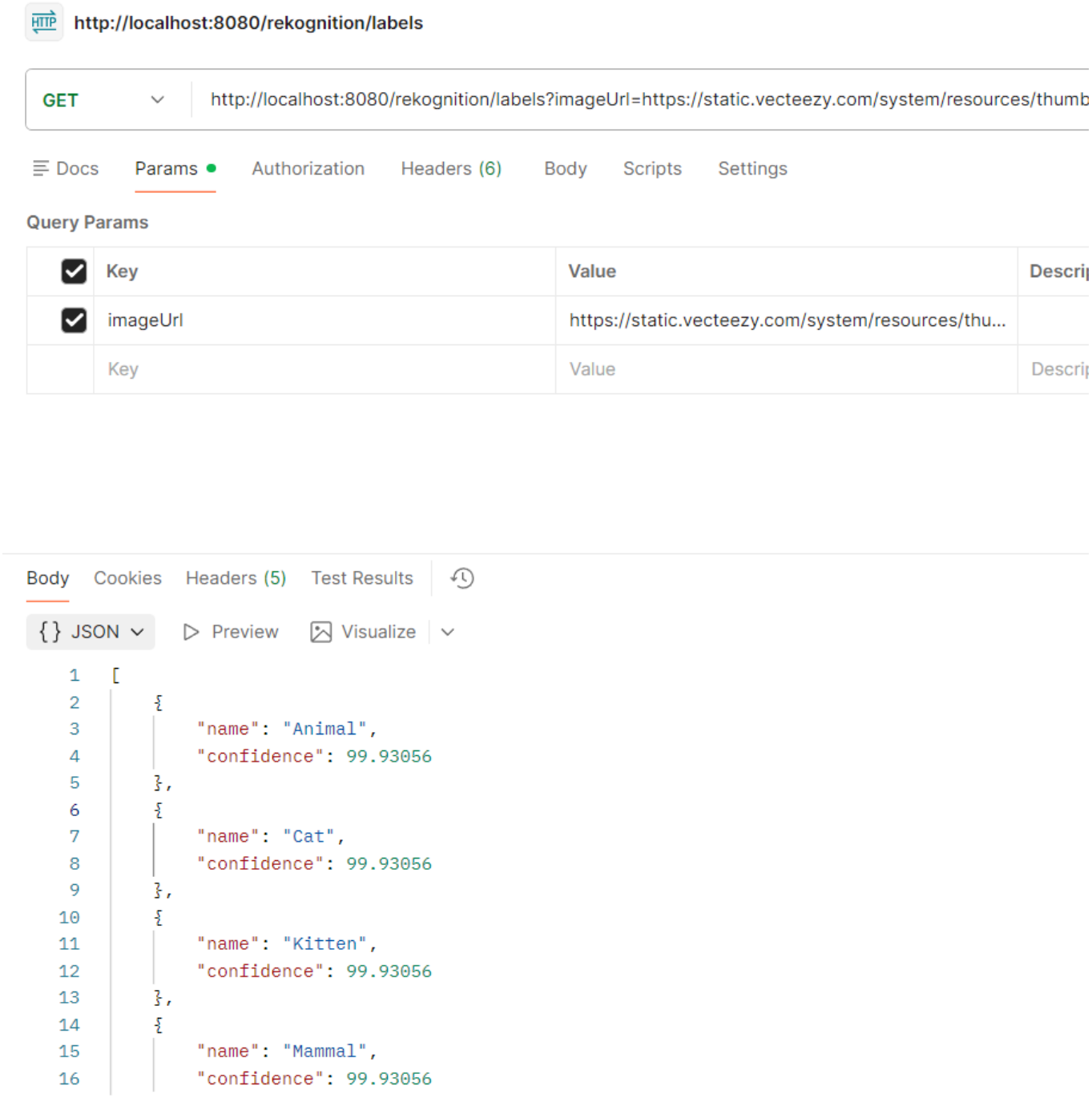


2- Deney Spring Boot ve IntelliJ IDEA kullanılarak yapılacak.

3- Spring projesi oluşturuldu ve gerekli dependencyler eklendi

4- Controller ve Service sınıfları oluşturuldu.

5- Postman kullanılarak kedi fotoğrafı test olarak gönderildi.



Amazon Comprehend

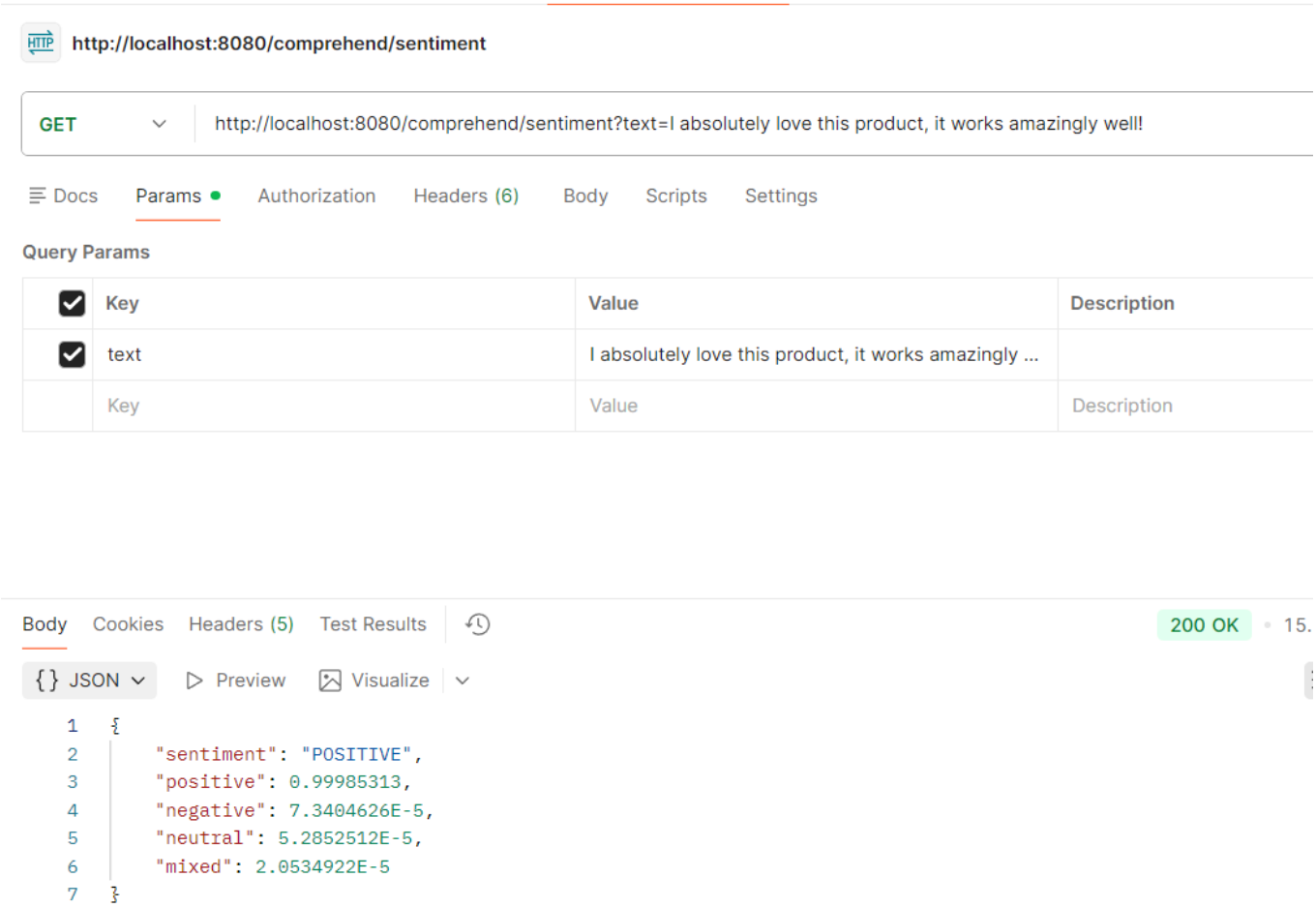
1- Test user'a readonly servis permissonı verildi.

2- Servis dependencyleri projeye eklendi.

3- Service ve controller sınıfları ve response entity sınıfları oluşturuldu ve implement edildi.

4- Sentiment testi için Postman üzerinden url GET kullanılır ve sonuç alınır:

URL:"http://localhost:8080/comprehend/sentiment?text=I absolutely love this product, it works amazingly well!"



5- Entity testi için de aynı şekilde entity endpointine ulaşılır.
URL: http://localhost:8080/comprehend/entities?text=Salvador Dali was born in May 11 1904 in

Catalonia

Body Cookies Headers (5) Test Results

{} JSON Preview Visualize


```
[
    {
        "text": "Salvador Dali",
        "type": "PERSON",
        "confidence": 0.9994832
    },
    {
        "text": "May 11 1904",
        "type": "DATE",
        "confidence": 0.9993491
    },
    {
        "text": "Catalonia",
        "type": "LOCATION",
        "confidence": 0.9990953
    }
]
```

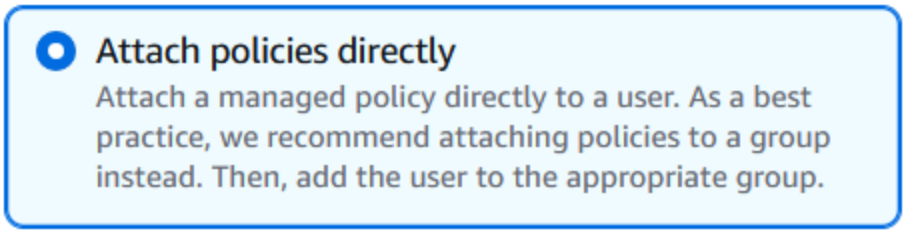

Tablo 2 - AI Servisleri Performans Değerlendirmesi

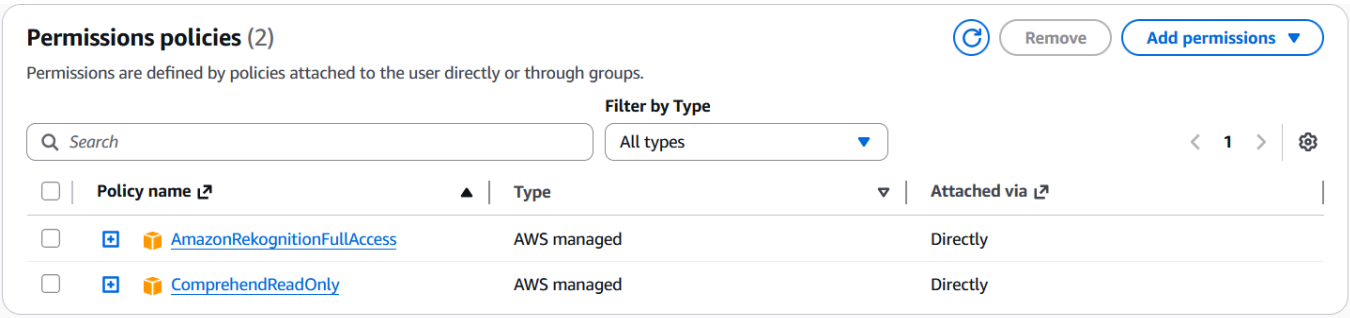
Ölçüt	Rekognito	Comprehend	Sebep/ Açıklama
Kurulum Süresi	35dk	25dk	Kod yazmadan önce usera yetki vermek ve dependency eklemek yeterli.
Kullanım Kolaylığı	4	5	İkisi de gayet kolay ancak Rekognito imagebyte'a dönüştürme istiyor.
Ölçeklenebilirlik	5	5	ikisi de aynı AWS güvenlik altyapısını paylaşıyor.
API Entegrasyon Kolaylığı	4	5	Comprehend API'ı daha sade
Güvenlik seviyesi	5	5	ikisi de aynı AWS güvenlik altyapısını paylaşıyor.
Normalize Puan	**9.38**	**12.5**	

Görev–3: Siber Güvenlik Servisleri Nicel Analizi (25 Puan)

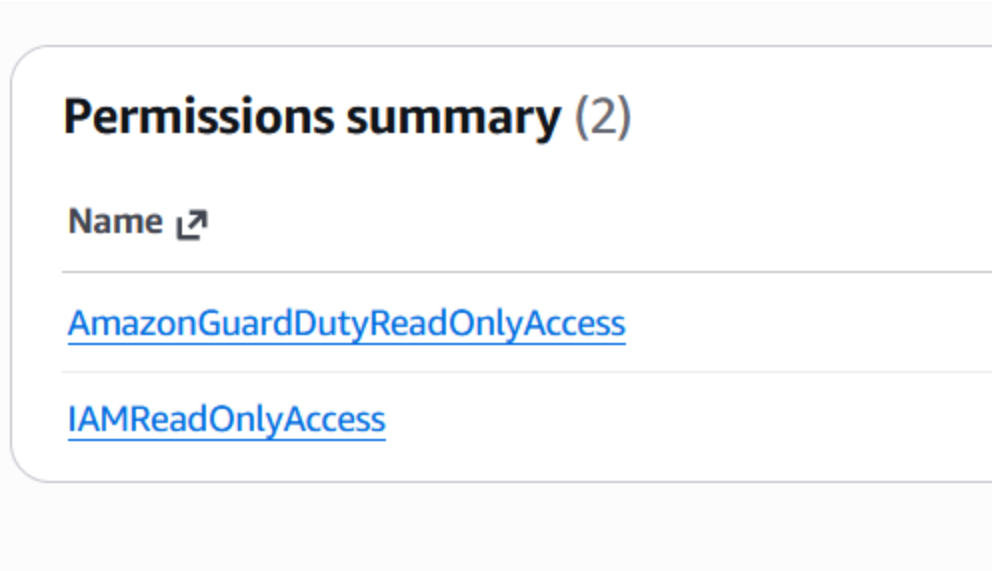
IAM ve GuardDuty

1- Bu demo projesine ilk başladığımızda oluşturduğumuz user IAM servisi kullanılarak oluşturulmuştu. Bu bir öğrenme projesi olduğu için Eklediğimiz servislerin permissonlarını kullanıcıya direkt olarak verdim ve grup oluşturmadım.





2- IAm test kullanıcısına gerekli izinleri ekledim.

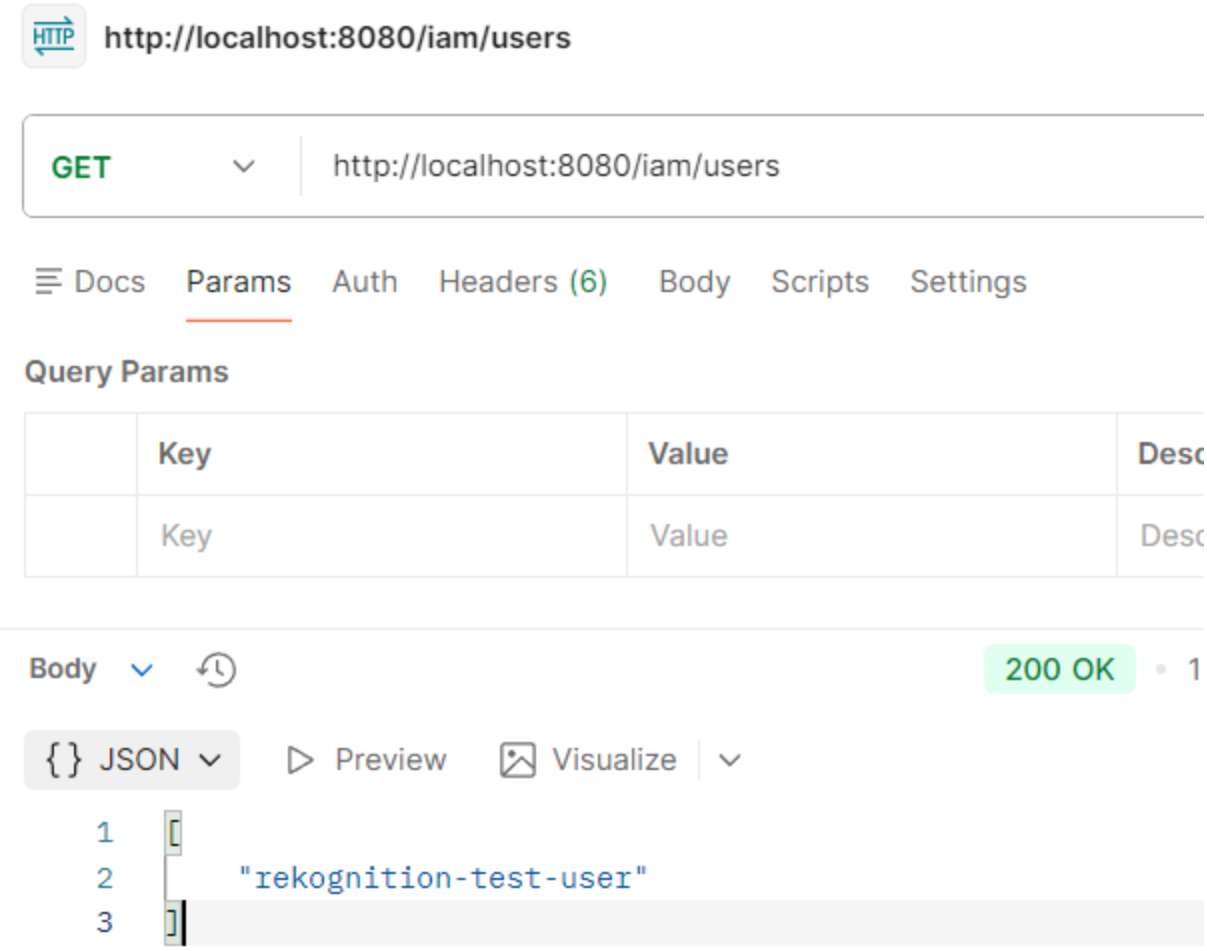


3- Servisi gözlemlenebilir hale getirmek için Spring projeme bu iki servisi dependency olarak ekledim.

4- AWSConfig classı oluşturdum ve bu config classını IamClient ve GuardDutyClient oluşturmak için kullandım.

5- Controller classlarını oluşturdum ve endpointlerini tanımladım.

6- Postman üzerinden endpointler test edildi.
http://localhost:8080/iam/users



GET http://localhost:8080/iam/policies
GET http://localhost:8080/iam/users/{{username}}/policies
GET http://localhost:8080/iam/roles
GET http://localhost:8080/guardduty/detector
GET http://localhost:8080/guardduty/findings

Tablo 3. Siber Güvenlik Servisleri Değerlendirme Tablosu

Ölçüt	IAm	GuardDuty	Sebep / Açıklama
Yetki Yönetme Esnekliği	5	2	IAm policy/role/group sistemine sahip.
Loglama ve izlenebilirlik	3	5	IAm Loglama için CloudTrail gerektiriyor.
Tehdit Algılama Yeteneği	2	5	GuardDuty ML tabanlı anomali tespitine sahip
Entegrasyon Kolaylığı	4	4	Consoldan enable ettikten sonra tek Bean ile bağlanabiliyor.
Varsayılan Güvenlik Seviyesi	4	5	GuardDuty herhangi bir configuration olmadan korumaya başlıyor.
Normalize Puan	9	10.5	

Görev–4: Yazılım Geliştirme Servisleri Nicel Analizi (20 Puan)

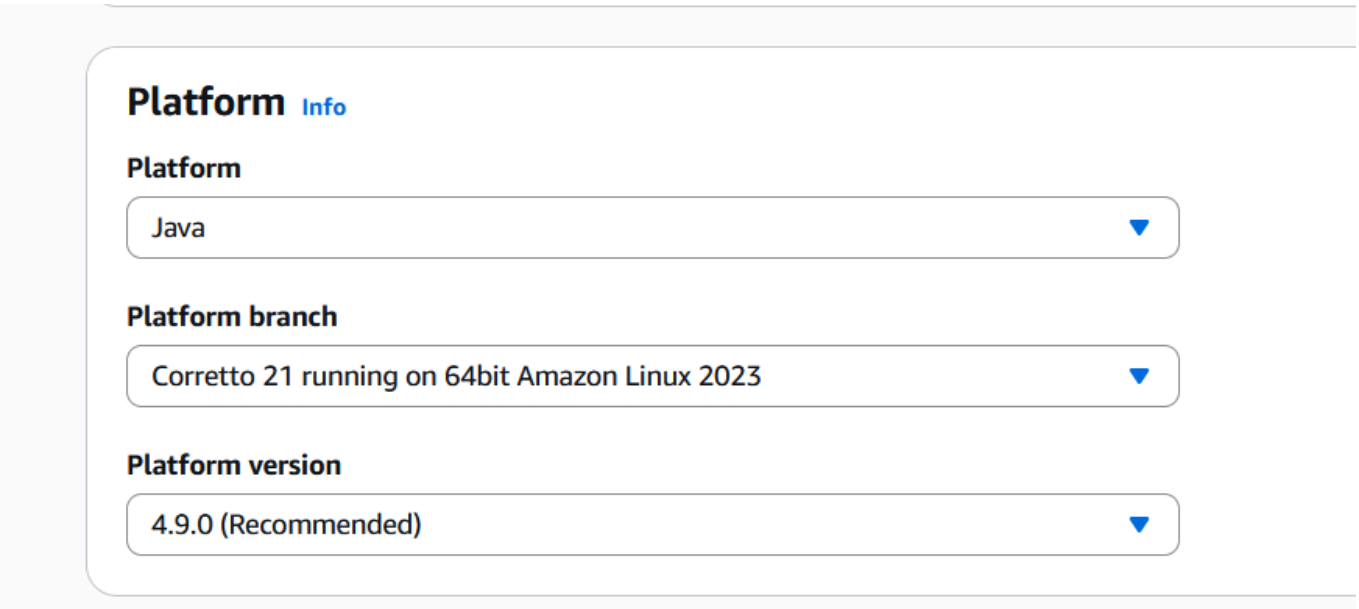
Elastic Beanstalk

Elastic Beanstalk uygulamamız için bir host platform olacak.

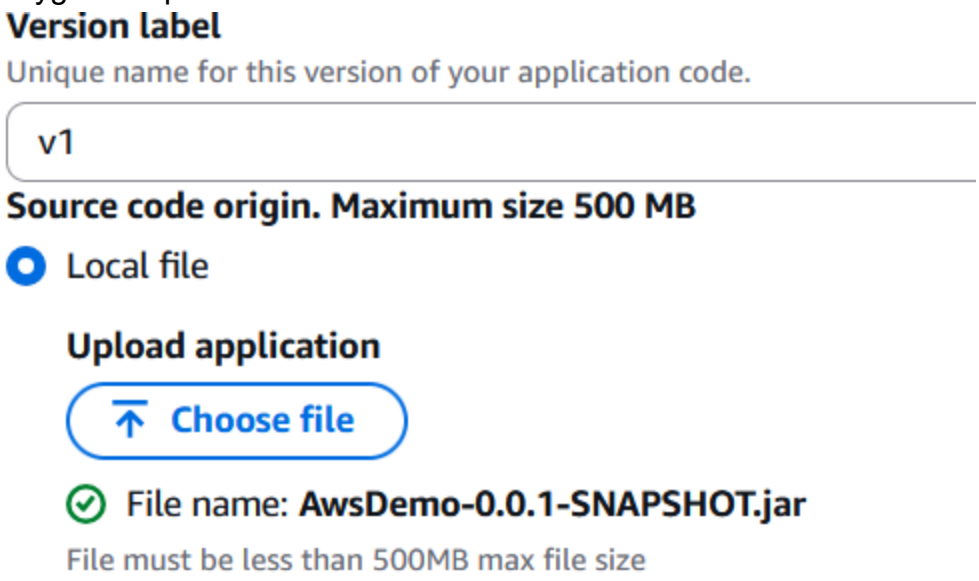
1- Gerekli dependency eklendi.
2- AWSConfig classında Client Bean oluşturuldu.
3- BeanstalkController sınıfı oluşturuldu.
4- Maven kullanılarak DemoApp'i .jar dosyası olarak paketlendi.

_remote.repositories	3/11/2026 6:13 PM	REPOSITORIES File	1 KB
AwsDemo-0.0.1-SNAPSHOT	3/11/2026 6:13 PM	Executable Jar File	43,848 KB
AwsDemo-0.0.1-SNAPSHOT.pom	3/11/2026 6:12 PM	POM File	3 KB
maven-metadata-local	3/11/2026 6:13 PM	Microsoft Edge HT...	1 KB

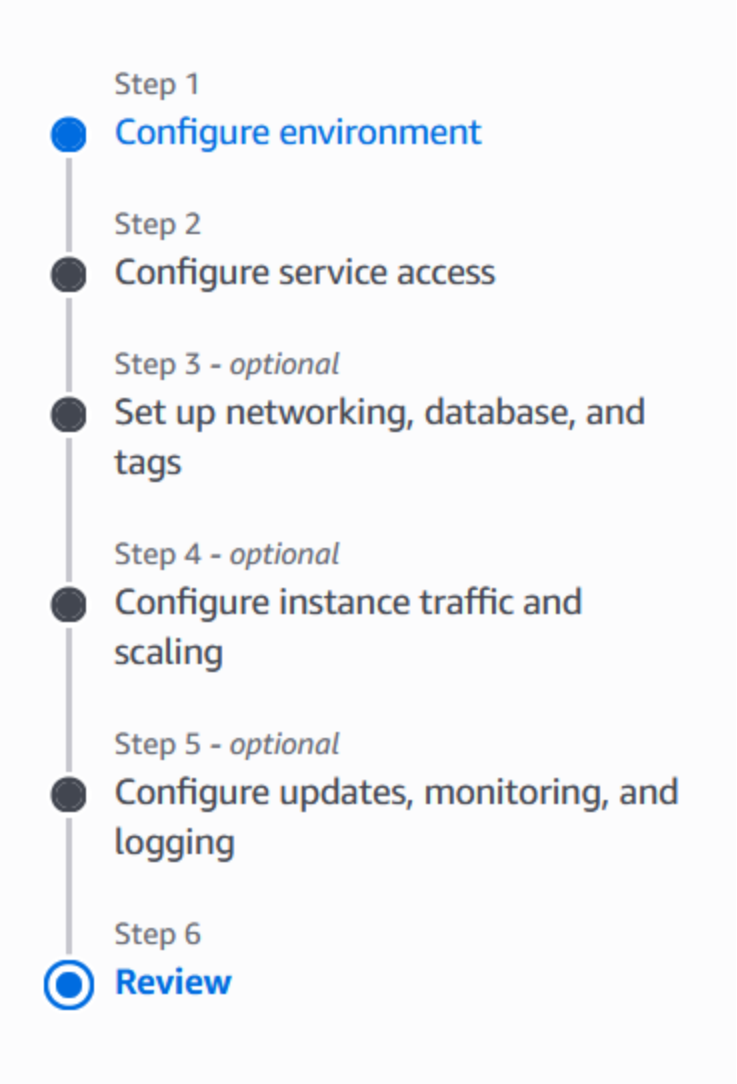
3- AWS üzerinden bir application ve bu application içinde gerekli özelliklere sahip bir environment oluşturuldu.



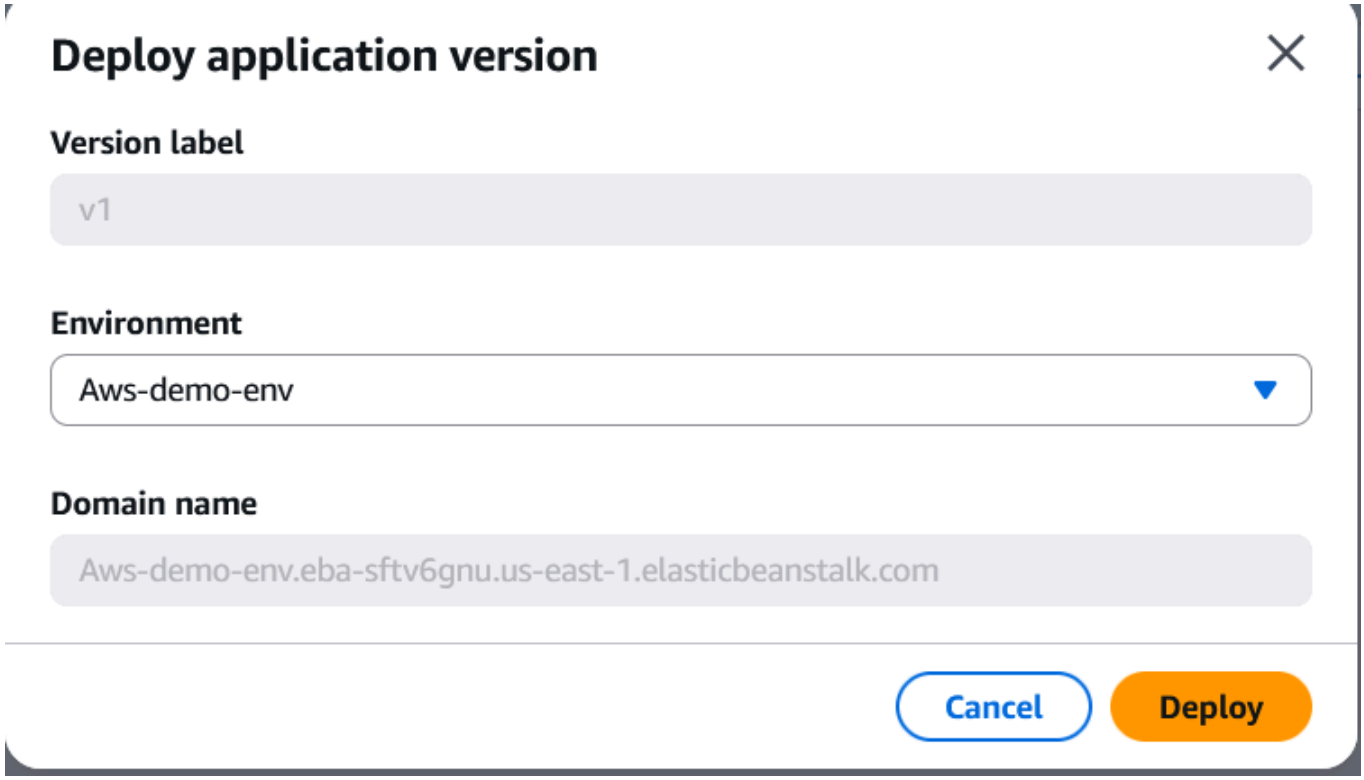
5- Uygulama platforma eklendi



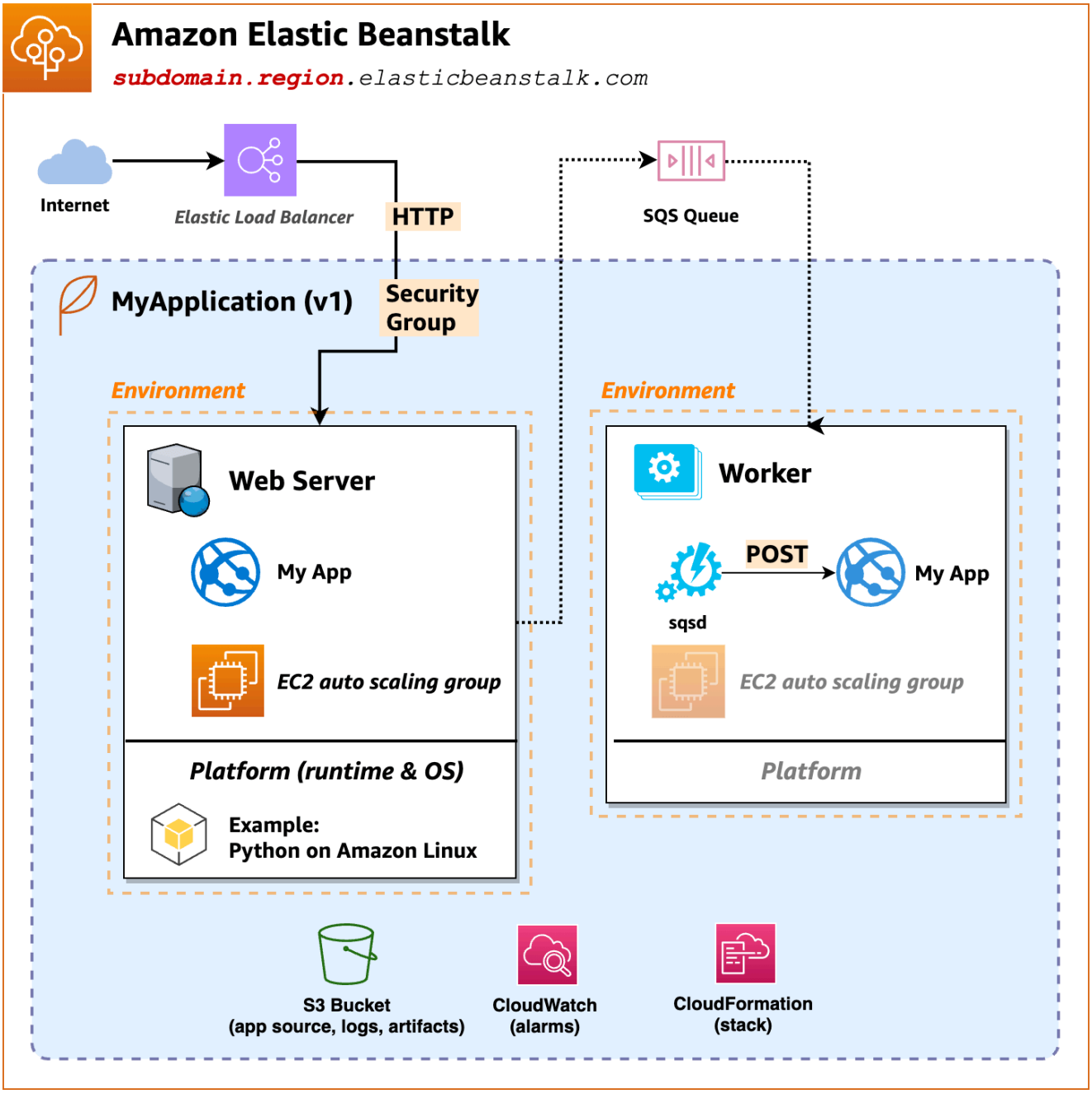
6- Enviroment için gerekli adımlar tamamlandı.



7- Uygulama Cloud'a deploy edildi.

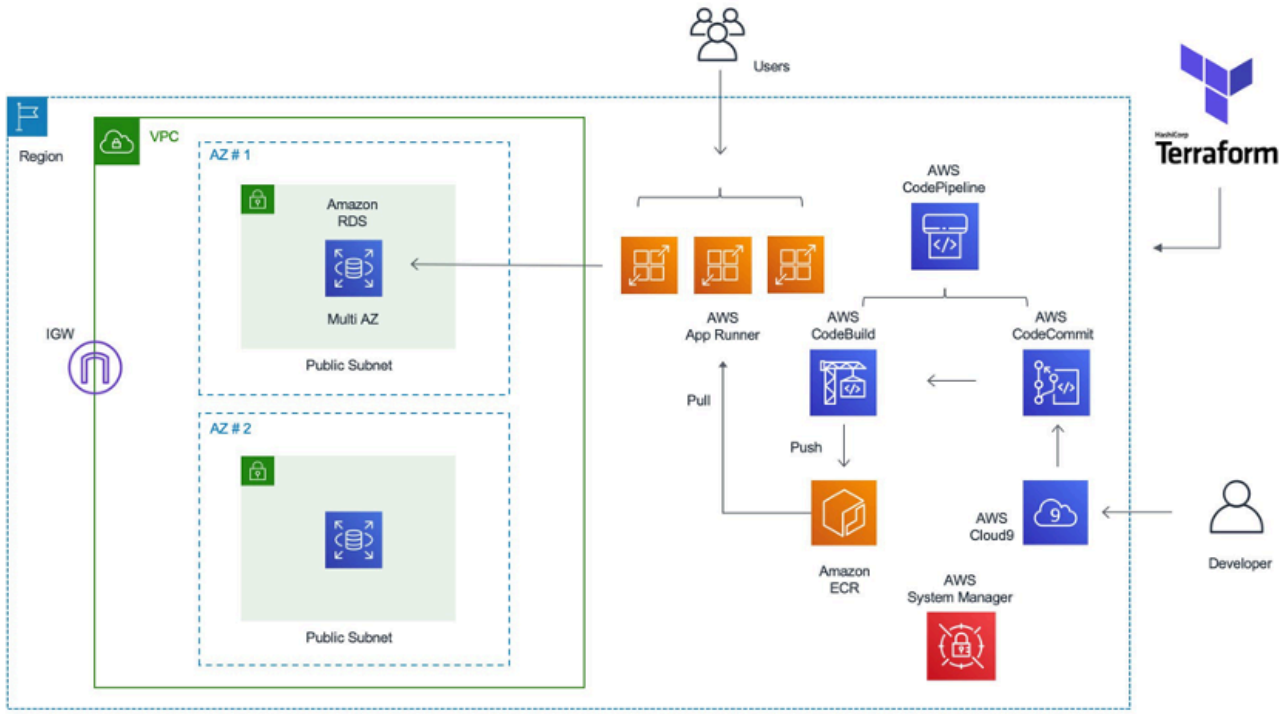


Bu adımların sonunda AWS tarafından bize sağlanan demo domain üzerinden projenin web üzerinde nasıl göründüğünü gözlemledik.



CodePipeline

CodePipeline'ı uygulamamızda kullanarak build, test ve deployment kısımlarını otomatikleştirebiliriz.

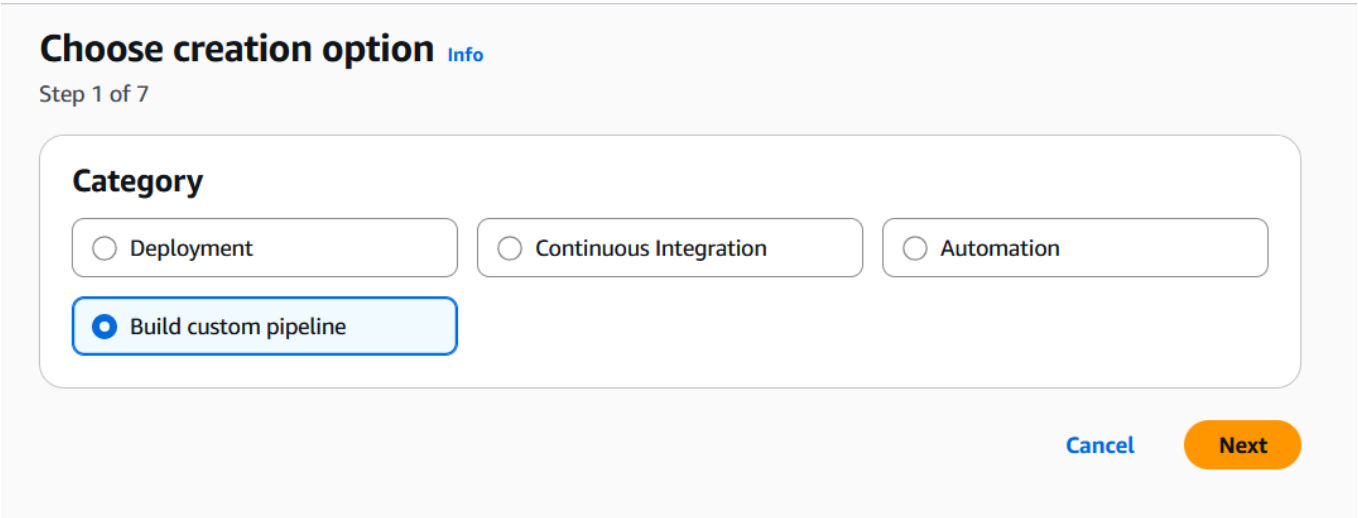


Codepipeline için seçtiğimiz setup şu şekilde olacak:
GitHub → CodeBuild with Maven → Beanstalk

Bu yüzden başlamadan önce aktif bir Elastic Beanstalk eviromentimiz ve kullanacağımız projenin Github üzerinde kullanılabilir bir sürümü olması gerekmekte.

1- İlk olarak AWS Console üzerinden CodePipline toolunu seçerek yeni pipeline üretmemiz gerekiyor.

Burada seçeneklerden Build custom pipeline seçiyorum çünkü setupımız oldukça net ve adımları tek tek görmek bu deneyi daha net anlamam için önemli.



2- Pipeline settinglerini düzenliyoruz.

Step 2 of 7

Pipeline settings

Pipeline name

Enter the pipeline name. You cannot edit the pipeline name after it is created.

aws-demo-pipeline

No more than 100 characters

Execution mode Info

Choose the execution mode for your pipeline. This determines how the pipeline is run.

- ○ Superseded
- ◉ Queued
- ○ Parallel

Service role

- ◉ New service role
  Create a service role in your account
- ○ Existing service role
  Choose an existing service role from your account

Role name

AWSCodePipelineServiceRole-us-east-1-aws-demo-pipeline

Type your service role name

☑ Allow AWS CodePipeline to create a service role so it can be used with this new pipeline

3- Uygulamamızın pipeline'a aktarılması için GitHub ile bağlantısını sağlıyoruz.

Add source stage Info

Step 3 of 7

Source

Source provider

This is where you stored your input artifacts for your pipeline. Choose the provider and then provide the connection details.

GitHub (via OAuth app)

Grant AWS CodePipeline access to your GitHub repository. This allows AWS CodePipeline to upload commits from GitHub to your pipeline.

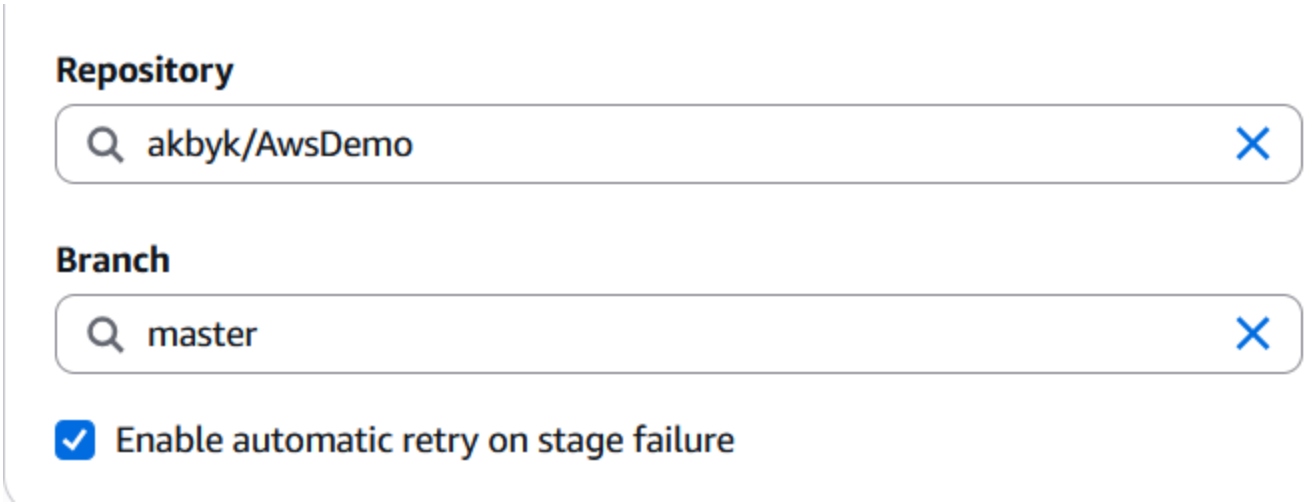
Connect to GitHub

The GitHub (via OAuth app) action is not recommended

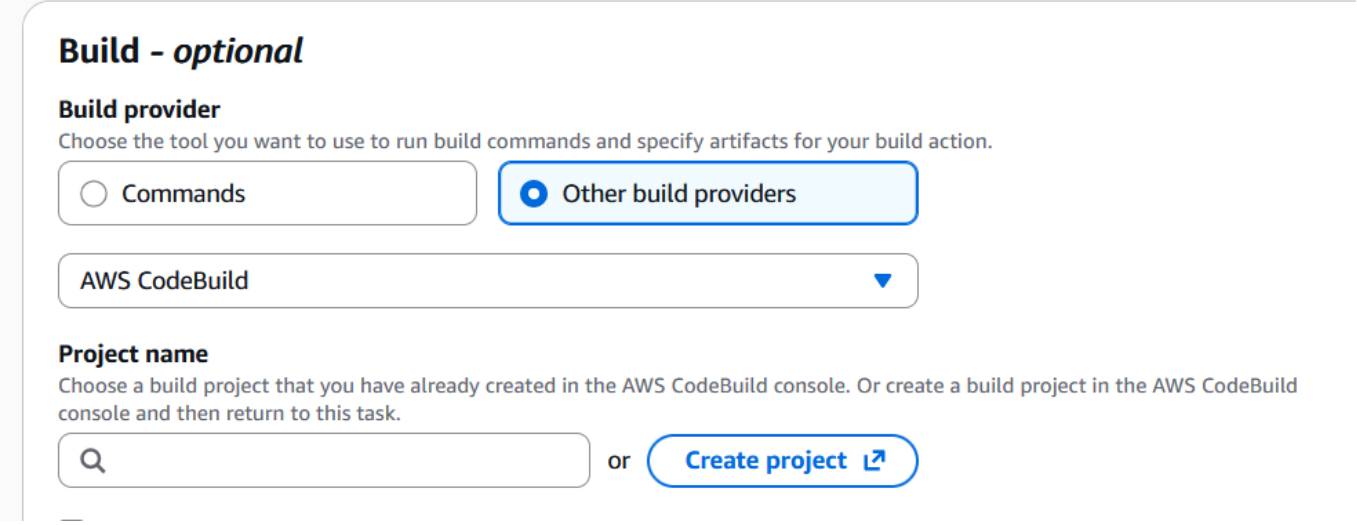
The selected action uses OAuth apps to access your GitHub repository. This is no longer the recommended method. Instead, choose the GitHub (via GitHub App) action to access your repository by creating a connection. Connections use GitHub Apps to manage authentication and can be shared with other resources. Learn more

☑ Enable automatic retry on stage failure

Bağlantı sağlandıktan sonra programın bulunduğu repo ve branchi seçiyoruz.



4- Build aşamasına geldiğimizde burada kullanılmak için bir CodeBuild projesi oluşturuyoruz.



Oluşturduğumuz bu projede kendi build commandlerimiz ekliyoruz.

Build commands Info

```yaml
version: 0.2
phases:
  install:
    runtime-versions:
      java: corretto21
  build:
    commands:
      - mvn clean package -DskipTests
artifacts:
  files:
    - target/aws-demo-0.0.1-SNAPSHOT.jar
  discard-paths: no
```

Successfully created aws-demo-build in CodeBuild.

Bu deney kapsamında olmadığı için test kısmını geçiyoruz.

Add test stage - *optional* Info

Step 5 of 7

Test - *optional*

Test provider

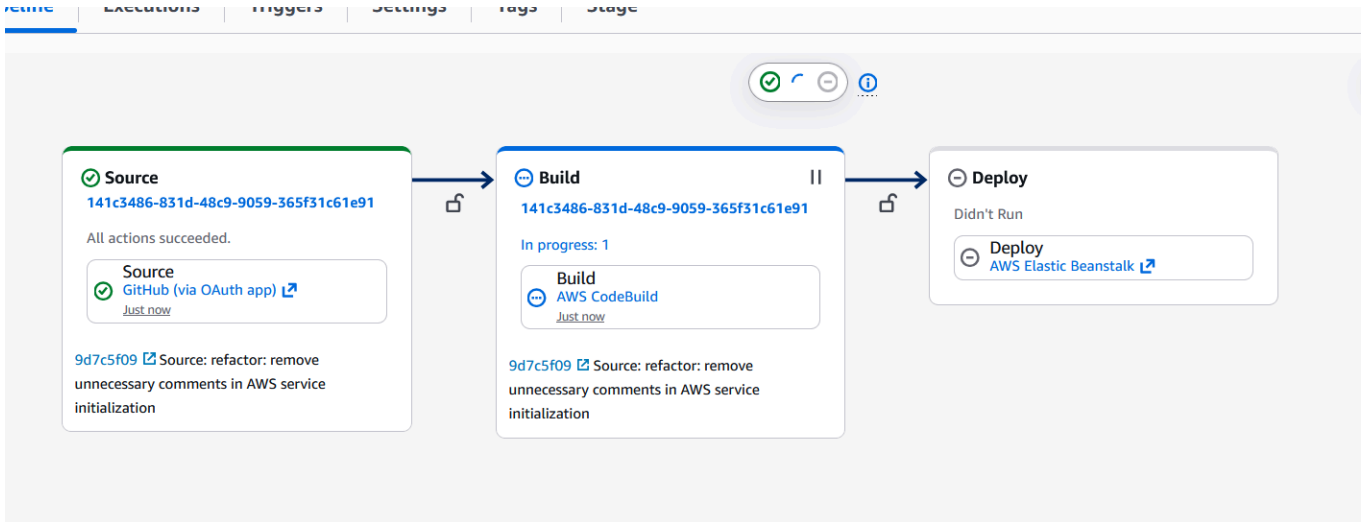
Choose how you want to test your application or content. Choose the provider, and then provide the configuration details for th provider.

Enable automatic retry on stage failure

Cancel | Previous | Skip test stage

5- Deployment kısmına geçtiğimizde Elastic Beanstalk seçiyoruz

Application ve enviroment için Elastic Beanstalk kurulumunu yaparken oluşturduklarımızı seçiyoruz.
Lifecycleını konsoldan izleyebiliriz.



Deneyin bu kısmında Build phaseinde bir error alıyoruz

Error message

Build terminated with state: FAILED

Failed phase

UPLOAD_ARTIFACTS

Failed phase code

CLIENT_ERROR

Failed phase message

no matching artifact paths found

Debuglamak için öncelikle loglara bakıyoruz

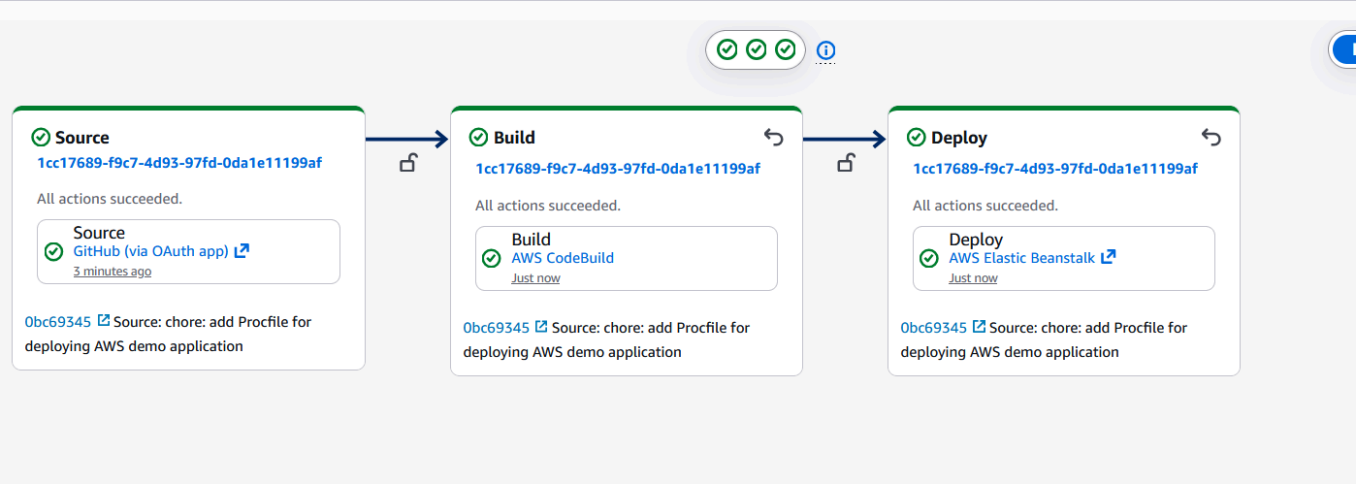
```
[INFO] ------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------
[INFO] Total time:  17.355 s
[INFO] Finished at: 2026-03-16T11:00:51Z
[INFO] ------------------------------------------
```

Kodun kendisinin buildinde herhangi bir sorun olmadığını görüyoruz.

Artifactlarda problem olduğu için build aşamasına geri dönüyoruz ve build commandleri override ederek düzeltiyoruz. Eklediğmiz artifact isminin doğruluğunu pom.xml dosyasından kontrol ediyoruz.

Düzenlemeyi kaydettikten sonra "Release Pipeline" kullanarak tekrar deniyoruz.
Bu sefer Build aşamasında sorun yaşamadık ancak deploy kısmı fail mesajı verdi. Elastic Beanstalk loglarını gözden geçirerek sorunun Procfile eklemememden kaynaklandığını görüyorum. Gerekli bilgileri ekleyip buildspec dosyasını düzenledikten sonra pipeline'ı tekrar çalıştırıyorum.

Program üzerinde yaptığım değişiklikleri ve eklediğim dosyaları Github'a push ettiğimde CodePipeline bunu otomatık olarak algılıyor ve release triggerlıyor.



CodePipeline kurulumumuz burada bitiyor.

Tablo 4. Yazılım / Developer Servisleri Değerlendirmesi

Ölçüt	Elastic Beanstalk	CodePipeline	Sebep/Açıklama
Kurulum Süresi	20	30	
CI/CD Desteği	2	5	Beanstalk'ta CI yok.
Otomasyon Seviyesi	4	5	CodePipeline tam otomatik. Beanstalk için Procfile manuel
Sürüm Yönetimi Uyumu	4	5	CodePipeline'da Git Branchı direkt pipeline'a bağlıyoruz.
Bulut Bağımlılığı	5	4	CodePipeline Github Actions ile değiştirilebilir.
Normalize Puan	7.5	10	

Görev–5: Nicel Veriye Dayalı Analitik Yorum (10 Puan)

Görev 2,3 ve 4’ten elde edilen sonuçlara bağlı üç yorum.

- Cloud servisleri bize sadece veriyi uzakta depolama seçeneği vermekle kalmaz aynı zamanda bir çok farklı amaçla kullanılabilecek servisler de sağlar. Bu servisleri kullanarak geliştirme sürecimizi aynı zamanda hızlandırır, daha güvenli hale getirir ve zenginleştiririz.

- Aynı alanda incelediğimiz farklı servisler birbirine göre düşük/yüksek puan alsa da sadece bu tablolara bakarak ne kullanmamız gerektiğini seçemeyiz. İhtiyacımız olan güvenlik seviyesi, maliyet ve diğer çeşitli ihtiyaçları da hesaba katmak gerekir.
    - Ayrıca bir çok servisi tek başına ele almamız, birbirinden bağımsız düşünmemiz mümkün değil. Bu demo proje boyunda AWS'nin birbirini destekleyen yapı taşlarından oluşan bir bütün olduğunu gözlemledik ve bit bütün olarak uyguladık.
        - Örneğin projenin büyük bir kısmında IAM servisini kullanmamız sağlam bir güvenlik yapısı oluşturabilmemiz için bize fırsat tanıdı.
- Cloud Computing sistemlerinin öğrenilmesi özellikle web alanında kariyerini geliştirmek isteyen her öğrenci ve yeni başlayan için hayati bir önem taşıyor. Bu sistemlerin ölçeklenebilirliği, tablolarda da olduğu gibi çok yüksek bu sebepten dolayı kişisel projelerden enterprise seviyesine kadar her alanda kullanım sunuyorlar.